

tf.compat.v1.assert_none_equal Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/assert_none_equal

Assert the condition $x \neq y$ holds element-wise.

Function Signatures

```
tf.compat.v1.assert_none_equal( x, y, data=None,
                                summarize=None, message=None, name=None )
```

Code Examples

```
tf.compat.v1.assert_none_equal(
    x, y, data=None, summarize=None, message=None, name=None
)
```

```
tf.compat.v1.assert_none_equal(
    x, y, data=None, summarize=None, message=None, name=None
)
```

```
tf.compat.v1.assert_none_equal(  
x=x, y=y, data=data, summarize=summarize,  
message=message, name=name)
```

```
tf.compat.v1.assert_none_equal(  
x=x, y=y, data=data, summarize=summarize,  
message=message, name=name)
```

```
tf.debugging.assert_none_equal(  
x=x, y=y, message=message,  
summarize=summarize, name=name)
```

```
tf.debugging.assert_none_equal(  
x=x, y=y, message=message,  
summarize=summarize, name=name)
```

```
g = tf.Graph()  
with g.as_default():  
a = tf.compat.v1.placeholder(tf.float32, [2])  
b = tf.compat.v1.placeholder(tf.float32, [2])  
result = tf.compat.v1.assert_none_equal(a, b,  
message='"a != b" does not hold for the given inputs')  
with tf.compat.v1.control_dependencies([result]):  
sum_node = a + b  
sess = tf.compat.v1.Session(graph=g)  
val = sess.run(sum_node, feed_dict={a: [1, 2], b:[2, 1]})
```

```
g = tf.Graph()
```

Documentation

Assert the condition $x \neq y$ holds element-wise.

View aliases

Compat aliases for migration

See

Migration guide for
more details.

`tf.compat.v1.debugging.assert_none_equal`

Migrate to TF2

`tf.compat.v1.assert_none_equal` is compatible with eager execution and `tf.function`.

Please use `tf.debugging.assert_none_equal` instead when migrating to TF2.

Apart

from data, all arguments are supported with the same argument name.

If you want to ensure the assert statements run before the potentially-invalid computation, please use `tf.control_dependencies`, as `tf.function` auto-control dependencies are insufficient for assert statements.

Structural Mapping to Native TF2

Before:

After:

TF1 & TF2 Usage Example

Description

This condition holds if for every pair of (possibly broadcast) elements $x[i]$, $y[i]$, we have $x[i] \neq y[i]$.

If both x and y are empty, this is trivially satisfied.

When running in graph mode, you should add a dependency on this operation to ensure that it runs. Example of adding a dependency to an operation:

Returns

Raises